

Predicting the Risk of Negative Reviews for Olist Orders

Early-warning binary classification model using operational, payment, product, and customer features.

96,361

delivered orders after
joins

21.07%

bad-review target rate

4

models compared



Logistic Regression

Chosen by Negative F1, not raw
accuracy

Libraries



Also Libraries

```
import os
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
)

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 100)
```



data

- olist_customers_dataset.csv
- olist_order_items_dataset.csv
- olist_order_payments_dataset.csv
- olist_order_reviews_dataset.csv
- olist_orders_dataset.csv
- olist_products_dataset.csv

Load data

```
DATA_DIR = Path("data")

reviews = pd.read_csv(DATA_DIR / "olist_order_reviews_dataset.csv")
orders = pd.read_csv(DATA_DIR / "olist_orders_dataset.csv")
items = pd.read_csv(DATA_DIR / "olist_order_items_dataset.csv")
payments = pd.read_csv(DATA_DIR / "olist_order_payments_dataset.csv")
products = pd.read_csv(DATA_DIR / "olist_products_dataset.csv")
customers = pd.read_csv(DATA_DIR / "olist_customers_dataset.csv")

datasets = {
    "reviews": reviews,
    "orders": orders,
    "items": items,
    "payments": payments,
    "products": products,
    "customers": customers,
}

for name, df in datasets.items():
    print(f"{name}: {df.shape}")
```

rows columns

reviews: (99224, 7)
orders: (99441, 8)
items: (112650, 7)
payments: (103886, 5)
products: (32951, 9)
customers: (99441, 5)

Checking the 10 Columns with the Highest Missing Value Rates in the DataFrame

reviews

review_comment_title	0.883415
review_comment_message	0.587025
review_id	0.000000
review_score	0.000000
order_id	0.000000
review_creation_date	0.000000
review_answer_timestamp	0.000000
dtype: float64	

orders

order_delivered_customer_date	0.029817
order_delivered_carrier_date	0.017930
order_approved_at	0.001609
order_id	0.000000
order_purchase_timestamp	0.000000
order_status	0.000000
customer_id	0.000000
order_estimated_delivery_date	0.000000
dtype: float64	

items

order_id	0.0
order_item_id	0.0
product_id	0.0
seller_id	0.0
shipping_limit_date	0.0
price	0.0
freight_value	0.0
dtype: float64	

payments

order_id	0.0
payment_sequential	0.0
payment_type	0.0
payment_installments	0.0
payment_value	0.0
dtype: float64	

Checking the 10 Columns with the Highest Missing Value Rates in the DataFrame

products

product_category_name	0.018512
product_description_lenght	0.018512
product_name_lenght	0.018512
product_photos_qty	0.018512
product_weight_g	0.000061
product_height_cm	0.000061
product_length_cm	0.000061
product_width_cm	0.000061
product_id	0.000000
dtype: float64	

customers

customer_id	0.0
customer_unique_id	0.0
customer_zip_code_prefix	0.0
customer_city	0.0
customer_state	0.0
dtype: float64	

The useful prediction is not “who is happy”; it is “who may complain.”

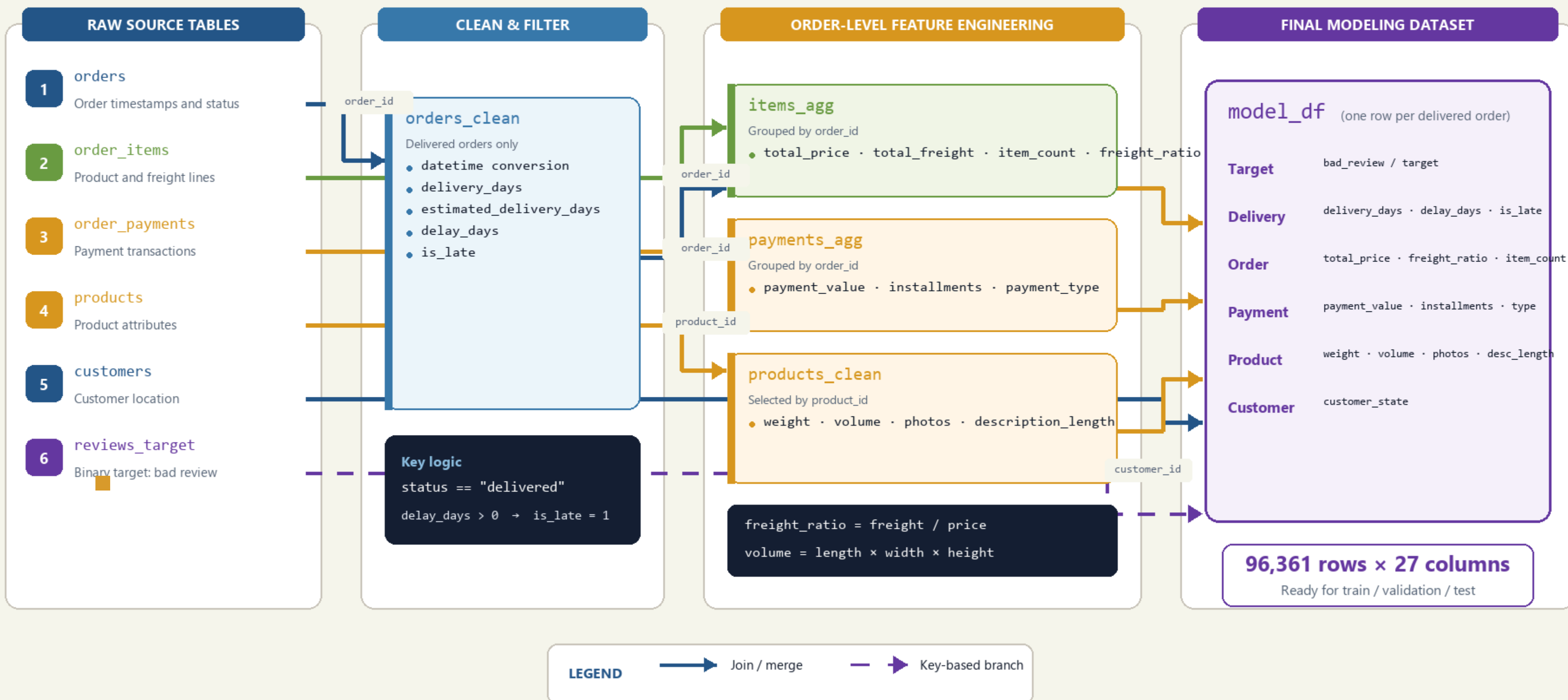
Olist needs a warning at delivery time, before the customer leaves a low review.

- Negative = review score 1–3
- Positive = review score 4–5
- Business priority: detect Negative

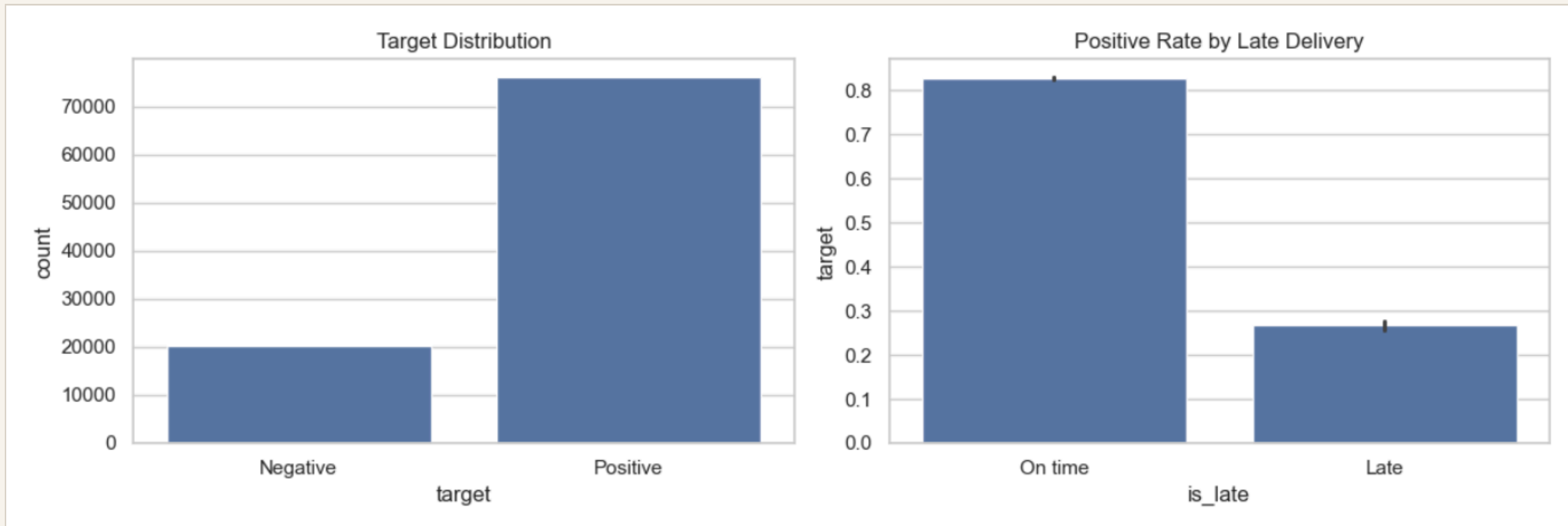
```
target
1      0.77068
0      0.22932
Name: ratio, dtype: float64
```

Building the Olist modeling dataset

Raw relational tables are cleaned, aggregated to order level, and joined into one ML-ready dataset.



Exploratory Data Analysis



Exploratory Data Analysis

```
numeric_preview = ["delivery_days", "delay_days", "total_price", "total_freight", "freight_ratio"]  
model_df.groupby("target")[numeric_preview].median()
```

	delivery_days	delay_days	total_price	total_freight	freight_ratio
target					
0	13.0	-9.0	89.97	18.19	0.234928
1	9.0	-13.0	84.90	16.86	0.223033

```
feature_cols = [  
    "delivery_days",  
    "estimated_delivery_days",  
    "delay_days",  
    "is_late",  
    "total_price",  
    "total_freight",  
    "item_count",  
    "freight_ratio",  
    "payment_value",  
    "payment_installments",  
    "product_weight_g",  
    "product_volume_cm3",  
    "product_photos_qty",  
    "product_description_lenght",  
    "payment_type",  
    "customer_state",  
]  
  
X = model_df[feature_cols].copy()  
y = model_df["target"].copy()  
,
```

Prepare Modeling Dataset

Numerical features:

- Fill null values with **median**
- Standardized using **StandardScaler**

Categorical features:

- Fill null values with the **most frequent value**
- **One-Hot** Encoding.

The dataset was split into **80% for training** and **20% for testing**, with stratification applied to preserve the class distribution of positive and negative reviews across both datasets.

Prepare Modeling Dataset

```
y_pred = clf.predict(X_test)
```

-> Pipeline tự động áp dụng cùng preprocessing đã học từ X_{train} lên X_{test} :

X_{test}

→ preprocessing đã fit trên X_{train}

→ `model.predict()`

Train and Compare Models

We trained 4 models:

1. Logistic Regression.

2. Decision Tree.

3. Random Forest.

4. Gradient Boosting.

	model	accuracy	positive_precision	positive_recall	positive_f1	negative_precision	negative_recall	negative_f1	roc_auc
0	Logistic Regression	0.750221	0.856623	0.820919	0.838391	0.419932	0.485475	0.450331	0.700601
2	Random Forest	0.821253	0.826906	0.978305	0.896257	0.741582	0.233136	0.354748	0.700898
3	Gradient Boosting	0.822965	0.824952	0.984616	0.897740	0.790698	0.217627	0.341313	0.708570
1	Decision Tree	0.713330	0.823903	0.809874	0.816829	0.330710	0.351797	0.340928	0.580892

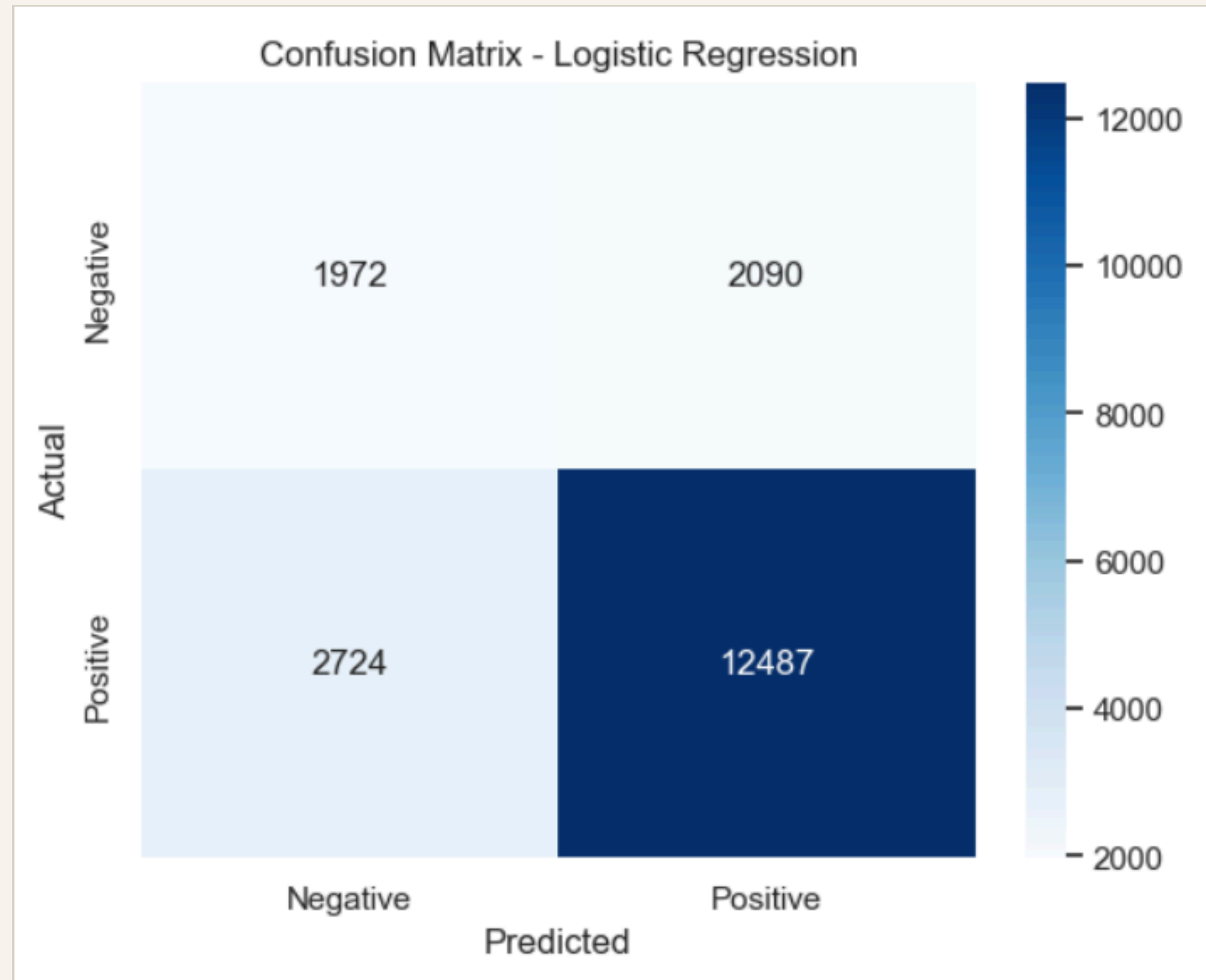
Best model for early warning:

Best model for early warning: Logistic Regression

Selection criterion: highest Negative F1, because Negative means risk of bad review.

	precision	recall	f1-score	support
Negative	0.42	0.49	0.45	4062
Positive	0.86	0.82	0.84	15211
accuracy			0.75	19273
macro avg	0.64	0.65	0.64	19273
weighted avg	0.76	0.75	0.76	19273

Best model for early warning:



Future Improvements

- **Tune the classification threshold to improve negative review detection.**
- **Apply SMOTE or undersampling to address class imbalance.**
- **Add seller-related and geographic features.**
- **Optimize Random Forest and Gradient Boosting through hyperparameter tuning.**

THANK YOU